

Especificação do Trabalho – Simplescalar (sim-outorder)

Objetivos:

- Uso do simulador simplescalar para investigar diferentes configurações de processadores superscalares e o impacto no desempenho.
- Os grupos serão compostos por no máximo 2 alunos.

Uso do Simplescalar (sim-outorder)

O Simplescalar é uma infraestrutura de software utilizada para realizar análise de desempenho de aplicações e microarquitetura (já sem suporte). O Simplescalar permite simular a execução de programas compilados em um conjunto de instruções similar ao MIPS (MIPS-like) chamado PISA. Podem ser simuladas aplicações (desde que recompiladas com o gcc) em diversas arquiteturas de processadores modernos.

Neste trabalho será utilizado a ferramenta *sim-outorder* que é um simulador detalhado de um processador superescalar com um sistema de memória de dois níveis e suporte a execução especulativa. O *sim-outorder* permite a configuração do core do processador, da hierarquia de memória e do predictor de desvios. Informações podem ser obtidas na sessão 4.4 do manual do usuário. O *sim-outorder* aceita (entre outros) os seguintes argumentos:

- **-fetch:ifqsize <size>** configura o tamanho do fila de *fetch* (instruções). Deve ser em potência de 2. Valor default igual a 4.
- **-decode:width <insts>** configura o tamanho da decodificação (instruções/ciclo). Deve ser em potência de 2. Valor default igual a 4.
- **-issue:width <insts>** configura o despacho máximo (instruções/ciclo). Deve ser em potência de 2. Valor default igual a 4.
- **-issue:inorder** força o simulador a despacho em ordem (in-order). O valor default é *false*.
- **-ruu:size <insts>** configura o tamanho da RUU (*Register Update Unit*) em instruções. Possui a funcionalidade de um buffer de reordenação (*Reorder Buffer*) e estações de reserva (*Reservation Stations*). Valor default igual a 16.
- **-commit:width <insts>** configura o tamanho do término das instruções (instruções/ciclo). Valor default igual a 4.
- **-lsq:size <insts>** configura a capacidade da fila de load/store (instruções) Valor default igual a 8.
- **-res:ialu <num>** configura a capacidade de ALUs inteiras. Valor default igual a 4.
- **-res:imult <num>** configura a capacidade de multiplicadores/divisores inteiros. Valor default igual a 1.
- **-res:fpalu <num>** configura a capacidade de FP ALUs (ponto flutuante). Valor default igual a 4.
- **-res: fpmult <num>** a capacidade de multiplicadores FP (ponto flutuante). Valor default igual a 1.
- **-res:mempport <num>** configura o número de portas da cache L1. Valor default igual a 2.

Um visão geral do pipeline do sim-outorder pode ser vista na Figura 1.

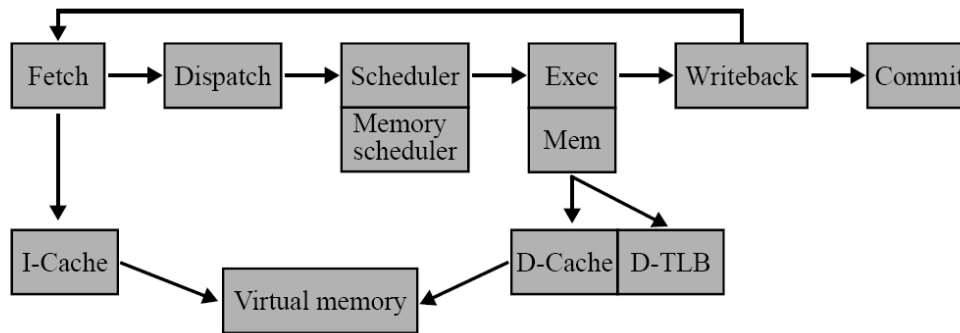


Figura 1. Pipeline do sim-outorder.

Além disso, é possível configurar o predictor de desvios através da opção *-bpred <type>*. O simulador possui 6 diferentes predictor, são eles:

- *nottaken* sempre prevê como não tomado;
- *taken* sempre prevê como tomado;
- *perfect* predictor perfeito;
- *bimod* predictor bimodal, utilizando uma BTB (*branch target buffer*) com 2-bit;
- *2lev* predictor adaptativo de 2 níveis;
- *comb* predictor combinado (bimodal and 2-level adaptive).

O default é um predictor bimodal de 2048 entradas.

Para instalar o simplescalar você necessita de um UNIX-like (Linux, Cygwin para Windows, etc). Após realizar o download do arquivo com o simulador (simplesim-3v0d.tgz) e descompactar será necessário configurar o simulador através do comando “make config-pisa” e “make”.

Realizar as simulações utilize os dois benchmarks selecionados para a sua dupla. Para executar o *sim-outorder* pode utilizar o seguinte comando:

sim-outorder -config <config file> <benchmark>

onde o *<config file>* possui as configurações do simulador e o *<benchmark>* é a aplicação que será executada (a aplicação poderá possuir parâmetros).

O arquivo com os benchmarks disponibilizado no moodle (benchmarks.zip) possui um exemplo do arquivo de configuração da cache e também possui os comandos para execução dos benchmarks.

A tabela abaixo apresenta a distribuição dos benchmarks por dupla de alunos.

Dupla de Alunos	Benchmarks
Ramiro Paz	GCC_1, IJPEGE_1
Tiago Mackedanz	GCC_2, IJPEGE_2
Mateus Schropfer	GCC_3, IJPEGE_3
Andressa Von Ahnt & Eduarda Gonçalves	GCC_4, VORTEX_1
Max Barn	LI_1, VORTEX_2
Maurício Leitzke	GO_1, IJPEGE_1
Frederico Bueno	PERL_1, IJPEGE_2
Edinilson Marques	GCC_1, VORTEX_1
Luiz Vitalino	LI_2, VORTEX_1
	LI_3, VORTEX_2

Tarefa 1 – Execução in-order e out-of-order

Verifique o efeito da execução em ordem e fora de ordem nos benchmarks (lembre-se que a opção default do simulador é fora de ordem).

Configurações para simulação em ordem e fora de ordem:

- Pipeline com largura de um (1) em ordem e fora de ordem;
- Pipeline com largura de um (2) em ordem e fora de ordem;
- Pipeline com largura de um (4) em ordem e fora de ordem;
- Pipeline com largura de um (8) em ordem e fora de ordem;

Pipeline com uma largura significa que os parâmetros de busca, decodificação, despacho e execução e término devem ser configurados.

Apresente:

- Apresente as medidas de total de ciclos executados e CPI para as seguintes configurações acima.
- Qual é o impacto em termos de CPI de permitir o processamento de mais instruções por ciclo ?
- Qual é o impacto em termos de CPI do uso de execução fora de ordem ?
- Um pipeline mais largo é mais efetivo para execução em ordem ou fora de ordem ?

Benchmark			
Parâmetros	Configuração 1	Configuração 2	Configuração n
<i>issue:inorder</i>			
<i>fetch:ifqsize</i>			
<i>decode:width</i>			
<i>issue:width</i>			
<i>ruu:size</i>			
<i>commit:width</i>			
<i>lsq:size</i>			
<i>res:ialu</i>			
<i>res:imult</i>			
<i>res:fpalu</i>			
<i>res:fpmult</i>			
<i>res:memports</i>			
Total de Ciclos			
CPI			

Tarefa 2 – Verificar o efeito do tamanho da janela

Verifique o efeito do tamanho da janela nos benchmarks selecionados. Configure a simulação fora de ordem e verifique o efeito do tamanho da janela (busca) em diferentes configurações do processador. Determine parâmetros adequados de decodificação e despacho contudo configure parâmetros fixos para execução e término.

Apresente:

- Apresente as medidas de total de ciclos executados e CPI para as seguintes configurações escolhidas.
- Qual é o impacto em termos de CPI ao utilizar janelas maiores ?

Tarefa 3 – Análise dos Previsores de Desvio

Verifique o efeito dos previsores de desvio no desempenho dos benchmarks escolhidos. Das seis opções do simulador apresente as estatísticas de apenas três possibilidades (nottaken, taken e bimod).

Apresente:

- Estatísticas de uso do predictor de desvios;
- Medida de CPI para verificar o quanto a previsão de desvios afeta no desempenho do processador (compare com a opção de predictor de desvios perfeito);
- Explique como funciona um predictor bimodal.
- Qual é o ganho relativo do uso de uma técnica mais elaborada como um predictor bimodal ?

Tarefa 4 – Customizar o seu processador para os seus benchmarks

Quando você projeta um processador, você constrói pensando no melhor compromisso (tradeoff) entre desempenho e custo. Ou seja, não adianta nada você possuir várias unidades de execução de FP se raramente (ou nunca) são todas utilizadas, porém para esta avaliação vários benchmarks são considerados.

Nesta tarefa você irá considerar apenas cada um dos seus benchmarks individualmente (e fará no máximo três diferentes configurações). Ou seja, você irá projetar dois processadores especializados (um para cada benchmark). Para isso devem ser considerados os parâmetros das tabelas abaixo.

Benchmark				
Parâmetros		Configuração 1	Configuração 2	Configuração 3
<i>fetch:ifqsize</i>				
<i>decode:width</i>				
<i>issue:width</i>				
<i>ruu:size</i>				
<i>commit:width</i>				
<i>lsq:size</i>				
<i>res:ialu</i>				
<i>res:imult</i>				
<i>res:fpalu</i>				
<i>res:fpmult</i>				
<i>res:memports</i>				
CPI				

Benchmark				
Parâmetros		Configuração 1	Configuração 2	Configuração 3
<i>fetch:ifqsize</i>				
<i>decode:width</i>				
<i>issue:width</i>				
<i>ruu:size</i>				
<i>commit:width</i>				
<i>lsq:size</i>				
<i>res:ialu</i>				
<i>res:imult</i>				
<i>res:fpalu</i>				
<i>res:fpmult</i>				
<i>res:memports</i>				
CPI				

Explique:

- Explique a razão da melhor configuração.
- Além do CPI (para desempenho) que parâmetros de custo devemos avaliar no projeto de um processador ?